CS 185/285 LLM RL Default Final Project

This repository contains the default LLM RL final project codebase.

The project studies RLHF for open-ended instruction following with: - base model: `Qwen/Qwen2.5-1.5B-Instruct` - benchmark: a curated `5k`-example dataset built from filtered WildChat prompts and LLM-ranked model-generated answers - primary metric: GPT-5.4 head-to-head win rate versus the frozen base model

The main Python package is: - `llm_rl_final_proj`

Repository Layout

- `llm_rl_final_proj/`: training, evaluation, and data code
- `scripts/modal_train.py`: Modal entrypoints for training and evaluation
- `dataset/`: the RLHF dataset to train on
- `public_eval/`: the repository evaluation assets used for local evaluation and Gradescope submissions

Environment

Initial setup:

```
uv run modal token new
uvx wandb login
```

For local head-to-head LLM evaluation on the repository evaluation files, also export your own OpenAI API key:

```
export OPENAI_API_KEY=...
```

Benchmark Dataset

The RLHF dataset is stored locally in: - `dataset/wildchat_min4_judged_5k_v1`

Split sizes: - `train_prefs = 4744` - `test_prefs = 256` - `train_gen = 4744` - `test_gen = 256`

Interpretation: - `train_prefs`: chosen/rejected preference pairs for offline preference optimization and reward-model training - `test_prefs`: held-out preference pairs for reward-model and offline diagnostics - `train_gen`: prompt-only split for online RLHF rollouts - `test_gen`: held-out prompt-only split for policy-vs-base evaluation

`train_prefs` and `train_gen` use the same prompt distribution. The difference is that `train_prefs` also includes paired responses and preference labels, while `train_gen` is the prompt source for online rollouts.

Benchmark Rules

When you run the required Part 1 methods or your Part 2 investigations, keep the benchmark setup fixed:

- do not train on additional prompt data or preference data beyond `dataset/wildchat_min4_judged_5k_v1`
- do not change the base model from `Qwen/Qwen2.5-1.5B-Instruct`
- keep `max_prompt_tokens = 700`
- for offline training and reward-model training, keep `max_response_tokens = 512`
- for online training and submission generation, keep `max_new_tokens = 256` and `max_response_tokens = 256`
- when building submission JSONLs, use deterministic decoding: `temperature = 0.0`, `top_p = 1.0`

Upload Benchmark Assets To Modal

Before training, upload the dataset to your Modal volume.

Upload the training dataset

```
        uv run modal volume put llm-rl-final-project-volume
dataset/wildchat_min4_judged_5k_v1 /synthetic_datasets/
```

This makes the dataset available inside Modal at: - `/vol/synthetic_datasets/wildchat_min4_judged_5k_v1`

Verify:

```
        uv run modal volume ls llm-rl-final-project-volume
/synthetic_datasets
```

Evaluation files

The evaluation files already live in this repository under: - `public_eval/`

When you run Modal entrypoints from this repo, the repository is mounted inside the container at: - `/root/project/`

So you do not need to upload `public_eval/` to your Modal volume. Use paths under: - `/root/project/public_eval/`

Project Part 1 Mandatory Methods

Offline: - `dpo` - `ipo` - `aot`

Online: - Bradley-Terry reward model - `grpo` - `drgrpo` - `gspo`

Typical Training Commands

Reward model

```
        uv run modal run scripts/modal_train.py::reward_model_train_remote -
- \
```

```
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1
\
          --train_split train_prefs \
          --eval_split test_prefs \
          --output_dir /vol/runs/wildchat_min4_judged_5k_reward_model_v1 \
          --per_device_train_batch_size 8 \
          --per_device_eval_batch_size 8 \
          --grad_accum_steps 4 \
          --lr 3e-5 \
          --num_train_epochs 3 \
          --max_prompt_tokens 700 \
          --max_response_tokens 512 \
          --eval_interval 25 \
          --save_interval 50 \
          --wandb_enabled \
          --wandb_project llm-rl-final-project \
          --wandb_name wildchat_min4_judged_5k_reward_model_v1
```

Offline preference training

```
        uv run modal run scripts/modal_train.py::train_remote -- \
          --algo dpo \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1
\
          --train_split train_prefs \
          --eval_split test_prefs \
          --generation_split test_gen \
          --output_dir /vol/runs/wildchat_min4_judged_5k_dpo_beta005_v1 \
          --beta 0.005 \
          --per_device_train_batch_size 4 \
          --per_device_eval_batch_size 4 \
          --grad_accum_steps 4 \
          --lr 5e-5 \
          --num_train_epochs 3 \
          --max_prompt_tokens 700 \
          --max_response_tokens 512 \
          --generation_eval_limit 32 \
          --generation_eval_max_new_tokens 256 \
          --generation_eval_every 100 \
          --eval_interval 100 \
          --save_interval 100 \
          --wandb_enabled \
          --wandb_project llm-rl-final-project \
          --wandb_name wildchat_min4_judged_5k_dpo_beta005_v1
```

Online RLHF training

```
        uv run modal run scripts/modal_train.py::rm_grpo_train_remote -- \
          --algo grpo \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1
\
          --train_split train_gen \
          --eval_split test_gen \
          --reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
          --reward_adapter_path
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000200/adapter
 \
          --output_dir /vol/runs/wildchat_min4_judged_5k_grpo_v1 \
          --steps 25 \
          --batch_size 16 \
          --group_size 4 \
          --min_new_tokens 32 \
          --max_new_tokens 256 \
```

```
  --temperature 0.8 \
  --top_p 0.95 \
  --lr 1e-5 \
  --grad_accum_steps 2 \
  --ppo_epochs 2 \
  --minibatch_size 8 \
  --clip_eps 0.2 \
  --kl_coef 0.01 \
  --max_prompt_tokens 700 \
  --max_response_tokens 256 \
  --eval_limit 32 \
  --eval_interval 25 \
  --save_interval 25 \
  --wandb_enabled \
  --wandb_project llm-rl-final-project \
  --wandb_name wildchat_min4_judged_5k_grpo_v1
```

W&B Sample Logging

The codebase already logs model generations to W&B.

During offline and online training, you should see: - a Markdown sample panel containing recent prompt/response examples - a `samples/generation_table` W&B table containing prompt text, reference response, model response, and, for online runs, reward-model score

This is implemented in: - `llm_rl_final_proj/train.py` - `llm_rl_final_proj/online/train_rm_grpo.py` - `llm_rl_final_proj/utils/wandb_utils.py`

For local evaluation during development, build the same JSONL submission files that you would upload to Gradescope and run the local autograder with your own OpenAI API key. The local autograder uses the same evaluation files and thresholds as Gradescope.

```
uv run python student_autograder/run_local_autograder.py \
  --submission_dir llm_rl_final_proj_public_submission \
  --output_json student_autograder_results.json
```

Gradescope Submission Builders

Upload JSONL outputs generated by running your trained models on the repository evaluation assets.

There are three submission categories: - Part 1 policy methods: generate on the repository `128`-prompt evaluation set - Part 1 reward model: score the repository `256` preference-pair set - Part 2 best methods: generate on the same repository `128`-prompt evaluation set

Part 1 policy submissions (`128` prompts)

For Part 1, the required policy methods are: - `dpo` - `ipo` - `aot` - `grpo` - `drgrpo` - `gspo`

Each of these should produce a JSONL built on: - `/root/project/public_eval/public_test_gen_prompts_128.jsonl`

Example:

```
uv run modal run
```

```
scripts/modal_train.py::build_policy_submission_remote -- \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --adapter_path /vol/runs/<run>/checkpoints/<step>/adapter \
          --prompts_jsonl
/root/project/public_eval/public_test_gen_prompts_128.jsonl \
          --output_jsonl /vol/submissions/dpo.jsonl \
          --max_prompt_tokens 700 \
          --max_new_tokens 256 \
          --temperature 0.0 \
          --top_p 1.0
```

Repeat this for each required Part 1 policy method, changing `--adapter_path` and `--output_jsonl` appropriately. Use the same 128-prompt file for all six Part 1 policy submissions.

Part 1 reward-model submission

The reward model you train is also autograded.

You must score the held-out preference pairs in: - `/root/project/public_eval/public_test_prefs_256.jsonl`

The autograder computes pair accuracy from the uploaded `chosen_score` and `rejected_score` values.

Command:

```
        uv run modal run
scripts/modal_train.py::build_reward_model_submission_remote -- \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --adapter_path
/vol/runs/<reward_model_run>/checkpoints/<step>/adapter \
          --prefs_jsonl
/root/project/public_eval/public_test_prefs_256.jsonl \
          --output_jsonl /vol/submissions/public_test_pref_scores.jsonl \
          --max_prompt_tokens 700 \
          --max_response_tokens 512
```

Part 2 policy submissions (128 prompts)

For Part 2, submit: - your strongest offline method as `offline_best.jsonl` - your strongest online method as `online_best.jsonl`

Both are built on: - `/root/project/public_eval/public_test_gen_prompts_128.jsonl`

Example offline Part 2 submission:

```
        uv run modal run
scripts/modal_train.py::build_policy_submission_remote -- \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --adapter_path
/vol/runs/<offline_part2_run>/checkpoints/<step>/adapter \
          --prompts_jsonl
/root/project/public_eval/public_test_gen_prompts_128.jsonl \
          --output_jsonl /vol/submissions/offline_best.jsonl \
          --max_prompt_tokens 700 \
          --max_new_tokens 256 \
          --temperature 0.0 \
          --top_p 1.0
```

Example online Part 2 submission:

```
        uv run modal run
```

```
scripts/modal_train.py::build_policy_submission_remote -- \
          --model_name Qwen/Qwen2.5-1.5B-Instruct \
          --adapter_path
/vol/runs/<online_part2_run>/checkpoints/<step>/adapter \
          --prompts_jsonl
/root/project/public_eval/public_test_gen_prompts_128.jsonl \
          --output_jsonl /vol/submissions/online_best.jsonl \
          --max_prompt_tokens 700 \
          --max_new_tokens 256 \
          --temperature 0.0 \
          --top_p 1.0
```

Download Submission Files From Modal

Create the local submission directory:

```
        mkdir -p llm_rl_final_proj_public_submission/policy_generations
        mkdir -p llm_rl_final_proj_public_submission/reward_model
        mkdir -p llm_rl_final_proj_public_submission/part2
```

Download Part 1 policy files:

```
        uv run modal volume get llm-rl-final-project-volume
/submissions/dpo.jsonl
llm_rl_final_proj_public_submission/policy_generations/
        uv run modal volume get llm-rl-final-project-volume
/submissions/ipo.jsonl
llm_rl_final_proj_public_submission/policy_generations/
        uv run modal volume get llm-rl-final-project-volume
/submissions/aot.jsonl
llm_rl_final_proj_public_submission/policy_generations/
        uv run modal volume get llm-rl-final-project-volume
/submissions/grpo.jsonl
llm_rl_final_proj_public_submission/policy_generations/
        uv run modal volume get llm-rl-final-project-volume
/submissions/drgrpo.jsonl
llm_rl_final_proj_public_submission/policy_generations/
        uv run modal volume get llm-rl-final-project-volume
/submissions/gspo.jsonl
llm_rl_final_proj_public_submission/policy_generations/
```

Download the reward-model file:

```
        uv run modal volume get llm-rl-final-project-volume
/submissions/public_test_pref_scores.jsonl
llm_rl_final_proj_public_submission/reward_model/
```

Download the Part 2 files:

```
        uv run modal volume get llm-rl-final-project-volume
/submissions/offline_best.jsonl
llm_rl_final_proj_public_submission/part2/
        uv run modal volume get llm-rl-final-project-volume
/submissions/online_best.jsonl
llm_rl_final_proj_public_submission/part2/
```

Zip the final submission for Gradescope:

```
        zip -r llm_rl_final_proj_public_submission.zip
llm_rl_final_proj_public_submission -x "*.DS_Store"
```

Autograder Files to Submit

Expected upload directory for the autograder:

```
llm_rl_final_proj_public_submission/
  policy_generations/
    dpo.jsonl
    ipo.jsonl
    aot.jsonl
    grpo.jsonl
    drgrpo.jsonl
    gspo.jsonl
  reward_model/
    public_test_pref_scores.jsonl
  part2/
    offline_best.jsonl
    online_best.jsonl
```

Autograder Thresholds

Gradescope and the local autograder use the same evaluation files and thresholds.

Thresholds you need to beat: - reward model pair accuracy: `0.74` - Part 1 DPO / IPO / AOT / GRPO / DrGRPO / GSPO: `0.68 / 0.55 / 0.72 / 0.68 / 0.64 / 0.60` - Part 2 offline best: `0.83` - Part 2 online best: `0.75`

For Part 2, you receive full credit on that portion if you clear either the offline threshold or the online threshold.